

DataShield — Руководство пользователя

Что такое DataShield

DataShield защищает небольшие файлы при передаче через ненадёжные каналы: радиоэфир, текстовые чаты, почту, носители с повреждениями. Файл кодируется в поток коротких самодостаточных пакетов (75 байт = 100 символов Base64), и приёмник восстанавливает его, даже если часть пакетов потеряна, продублирована или перекрыта мусором. Кодек работает в обе стороны: кодирование добавляет избыточность, декодирование накапливает и собирает исходный файл.

Требования

- .NET 10 (Desktop Runtime) или self-contained сборка приложения.
- Операционная система: Windows, Linux, macOS (GUI построен на Avalonia).
- Алгоритмы не требуют сети и работают полностью локально.

Подготовка файла к передаче (кодирование)

1. Выберите файл. Ограничения: размер до 4 МБ (точнее — до $(65\,535 - M) \cdot 64$ байт с ECC; поле размера допускает до 16 777 215 байт, но предел томов наступает раньше); длинное имя автоматически сокращается до 14 знаков с сохранением расширения (например, `documents.tar.gz` → `docume~.tar.gz`).
2. Задайте избыточность ECC, % — сколько избыточных пакетов добавить к пакетам данных. Значение по умолчанию 10%: восстановление гарантировано, если потеряно не более 10% секторов. Больше процент — надёжнее, но длиннее поток.
3. Выберите формат вывода:
 - **Текст (Base64)** — по одному пакету на строку; подходит для чатов, писем, текстовых файлов;
 - **Двоичный** — сырые 75-байтные пакеты; компактнее, для файловых каналов.
4. Нажмите «Кодировать». Прогресс показывает фазы: подготовка данных → вычисление ECC → формирование пакетов. Результат — готовый к копированию поток; копировать можно целиком или частями.

Кодер автоматически вставляет несколько копий заголовка (начало, конец и равномерно по потоку, ~3%), чтобы свойства файла встретились в потоке даже при сильных потерях.

Приём и восстановление (декодирование)

1. Вставьте полученный текст Base64 или загрузите двоичный файл; можно подавать поток частями и в любом порядке — сканер найдёт пакеты в шуме и удержит всё, что ещё не опознано.
2. По мере сканирования отображается: найденные файлы (заголовки), карта секторов каждого файла (принятые/пропущенные тома), счётчики подтверждений и коллизий.
3. Нажмите «Собрать». Если данных достаточно (все сектора на месте или потери покрыты ECC), файл восстанавливается и проверяется SHA-256 — результат гарантированно бит-в-бит исходному. Если данных мало, декодер честно откажет: соберите ещё пакетов и повторите.

Полезно знать:

- Пакеты можно передавать в произвольном порядке и с повторами — повторные копии лишь повышают счётчики подтверждений.
- Поток может содержать несколько файлов вперемешку — каждый собирается по своему заголовку.
- Секторы, пришедшие раньше заголовка своего файла, не теряются: декодер повторно привязывает их при появлении заголовка.

Гарантии восстановления

- Потеряно $\leq M$ секторов ($M \approx N \cdot \text{ECC}\%$): файл восстанавливается кодом Рида–Соломона.
- Все пакеты на месте: файл собирается напрямую.
- Подделка пакетов (чужие данные с корректным хешем практически невозможны): при равной правдоподобности версий декодер перебирает комбинации и принимает только ту, чей SHA-256 совпал с заголовком.
- Контрольная целостность: SHA-256 в заголовке; исход без совпадения хеша никогда не выдаётся.

Демонстрация и самопроверка

Поставляемый консольный стенд (`DataShield.Demo`) непрерывно генерирует случайные файлы, повреждает их случайно и проверяет восстановление. Индикаторы: **PASS** — восстановлено бит-в-бит; **WARN** — повреждение намеренно превышает запас прочности, отказ ожидаем; **FAIL** — сбой кодека (в штатной работе не встречается). Стенд удобно держать открытым для наблюдения за стабильностью и скоростью.

Краткая шпаргалка

Параметр	Значение
Пакет	75 байт = 100 символов Base64
Полезная нагрузка	64 байта на пакет
ЕСС по умолчанию	10% (восстановление до 10% потерь)
Копии заголовка	~3%, минимум 3 (начало/середина/конец)
Максимальный файл	~4 МБ без ЕСС, меньше при ЕСС
Имя файла	ASCII, до 14 знаков (длиннее — с «~»)
Верификация	SHA-256 + усечённый SHA-256 на каждый пакет